ROS2 - Python client library

Intelligent Robotics I

Andrés Suárez García

University of Vigo

Outline

Packages

what?

- directory with ros files
- ROS 2 packages organize code into reusable units
- camera handling, robot motion, etc.

workspace

- the workspace contains of ros packages
- contains source code
- folder name end “ws” by convention

create

```
## ros2 pkg create <package_name> --build-type ament_python

# ----- terminal 1 ----- #
cd ~/ros2_ws/src
ros2 pkg create my_package \
--build-type ament_python \
--node-name my_node

# ----------------------
going to create a new package
package name: my_package
destination directory: /home/user/ros2_ws/src
package format: 3

#...
cd my_package
ls
```

build

- build the package with colcon
- `packages-selec` selective package building

```
# ----- terminal 1 ----- #
cd ~/ros2_ws
colcon build --packages-select py_pkg
# load package scripts into workspace
source ~/ros2_ws/install/setup.bash
# run created node
ros2 run my_package my_node
# ----------------------
Hi from my_package.
```

package contents

- `my_package` python scripts
- `package.xml` package metadata (dependencies, license, etc.)
- `setup.py` script for building and installing
- `test` test files to verify the funcionality

package.xml

```
<?xml version="1.0"?>
<?xml-model
   href="http://download.ros.org/schema/package_format3.xsd"
   schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
 <name>my_package</name>
 <version>0.0.0</version>
 <description>TODO: Package description</description>
 <maintainer email="user@todo.todo">user</maintainer>

 <test_depend>ament_flake8</test_depend>
 <test_depend>python3-pytest</test_depend>

 <export>
   <build_type>ament_python</build_type>
 </export>
</package>
```

setup.py

```
from setuptools import setup
package_name = 'my_package'
setup(
 name=package_name,
 data_files=[
     ('share/ament_index/resource_index/packages',
             ['resource/' + package_name]),
     ('share/' + package_name, ['package.xml']),
   ],
 install_requires=['setuptools'],
 zip_safe=True,
 tests_require=['pytest'],
 entry_points={
     'console_scripts': [
             'my_node = my_py_pkg.my_node:main'
     ],
   },
)
```

launch 1/

Launch files are stored in `launch` directory

```
src/
  py_launch_example/
    launch/
    package.xml
    py_launch_example/
    resource/
    setup.cfg
    setup.py
    test/
```

`setup.py` has to be informed of the launch files

```
import os
from glob import glob
# Other imports ...

package_name = 'py_launch_example'

setup(
    # Other parameters ...
    data_files=[
        # ... Other data files
        # Include all launch files.
        (os.path.join('share', package_name, 'launch'),
   glob(os.path.join('launch', '*launch.[pxy][yma]*')))
    ]
)
```

launch 3/

- launch with `ros2 launch <package_name> <launch_file_name>`
- it is necessary to modify `package.xml`

```
<!-- add the following line -->
<exec_depend>ros2launch</exec_depend>
```

Node

build requirements

- write python script
- add dependencies into `package.xml`
- add entry point into `setup.py`

python node template

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class MyCustomNode(Node):

    def __init__(self):
        super().__init__("node_name")

def main(args=None):
    rclpy.init(args=args)
    node = MyCustomNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

Topic

setup

- packages should be create in `src`
- see Official Documentation link

```
cd ros2_ws/src
ros2 pkg create pubsub --build-type ament_python
```

publisher node 1/

Edit `ros2_ws/src/pubsub/pusub/minimal_publisher.py`

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class MinimalPublisher(Node):
    def __init__(self):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(String, 'topic', 10)
        timer_period = 0.5  # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0

class MinimalPublisher(Node):
    def timer_callback(self):
        msg = String()
        msg.data = f'Hello World: {i}'
        self.publisher_.publish(msg)
        self.get_logger().info(f'Publishing: "{msg.data}"')
        self.i += 1

def main(args=None):
    rclpy.init(args=args)
    minimal_publisher = MinimalPublisher()
    rclpy.spin(minimal_publisher)
    minimal_publisher.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Edit `ros2_ws/src/pubsub/pusub/minimal_suscriber.py`

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class MinimalSubscriber(Node):
    def __init__(self):
        super().__init__('minimal_subscriber')
        self.subscription = self.create_subscription(
            String, 'topic',
            self.listener_callback, 10)

    def listener_callback(self, msg):
        self.get_logger().info(f'I heard: "{msg.data}")

def main(args=None):
    rclpy.init(args=args)
    minimal_subscriber = MinimalSubscriber()
    rclpy.spin(minimal_subscriber)
    minimal_subscriber.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

dependencies and entry points

Add dependencies `package.xml`

```
<exec_depend>rclpy</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

Add entry point to `setup.py`

```
entry_points={
        'console_scripts': [
                'talker = pubsub.minimal_publisher:main',
                'listener = pubsub.minimal_suscriber:main',
        ],
}
```

build

```
cd ~/ros2_ws
# check dependencies
rosdep install -i --from-path src --rosdistro humble -y

# build
colcon build --packages-select pubsub

# add commands to environment
source install/setup.bash
```

- press `Ctrl + C` to stop node from spinning

```
# ----- terminal 1 ----- #
ros2 run pubsub talker
# ------------------------
[INFO] [minimal_publisher]: Publishing: "Hello World: 0"
[INFO] [minimal_publisher]: Publishing: "Hello World: 1"
[INFO] [minimal_publisher]: Publishing: "Hello World: 2"

# ------ terminal 2 ------ #
ros2 run pubsub listener
# -------------------------
[INFO] [minimal_subscriber]: I heard: "Hello World: 10"
[INFO] [minimal_subscriber]: I heard: "Hello World: 11"
[INFO] [minimal_subscriber]: I heard: "Hello World: 12"
```

Service

- `--dependencies` will add dependencies
- check `packages.xml`

```
cd ~/ros2_ws
ros2 pkg create srvcli \
--build-type ament-python \
--dependencies rclpy example_interfaces
```

- search `example_interfaces/srv/AddTwoInts.srv`

```
# request
int64 a
int64 b
---
# response
int64 sum
```

service node

- edit `ros2_ws/src/srvcli/srvcli/server_node.py`

```
#!/bin/bash python3
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class MinimalService(Node):
    def __init__(self):
        super().__init__('minimal_service')
        self.srv = self.create_service(AddTwoInts, 'add_two_ints',
               self.add_two_ints_callback)

    def add_two_ints_callback(self, request, response):
        response.sum = request.a + request.b
        self.get_logger().info(
    f'Incoming request a: {request.a} b: {request.b}
    )
        return response

def main():
    rclpy.init()
    minimal_service = MinimalService()
    rclpy.spin(minimal_service)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

client node

- go to `ros2_ws/src/srvcli/srvcli`
- edit `client_node.py`

```python
#!/bin/bash python3
import sys
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class MinimalClientAsync(Node):
    def __init__(self):
        super().__init__('minimal_client_async')
        self.cli = self.create_client(AddTwoInts, 'add_two_ints')
        while not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('service not available, waiting...')
        self.req = AddTwoInts.Request()

    def send_request(self, a, b):
        self.req.a = a
        self.req.b = b
        return self.cli.call_async(self.req)

def main():
    rclpy.init()
    minimal_client = MinimalClientAsync()
    future = minimal_client.send_request( int(sys.argv[1]), int(sys.argv[2]))
    rclpy.spin_until_future_complete(minimal_client, future)
    response = future.result()
    minimal_client.get_logger().info(
        'Result of add_two_ints: for %d + %d = %d' %
        (int(sys.argv[1]), int(sys.argv[2]), response.sum))
    minimal_client.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

entry points

Add entry point to `setup.py`

```
entry_points={
        'console_scripts': [
                'server = srvcli.server_node:main',
    'client = srvcli.client_node:main',
        ],
}
```

build

```
cd ~/ros2_ws
# check dependencies
rosdep install -i --from-path src --rosdistro humble -y

# build
colcon build --packages-select pubsub

# add commands to environment
source install/setup.bash
```

```
# ----- terminal 1 ----- #
ros2 run srvcli server

# ----- terminal 2 ----- #
ros2 run srvcli client 2 3
# ----------------------
[INFO] [minimal_client_async]: Result of add_two_ints: for 2 + 3 = 5

# ----- terminal 1 ----- #
# ----------------------
[INFO] [minimal_service]: Incoming request a: 2 b: 3
```

Interfaces

what?

- interface is structure of messages
- msg is interface for topics
- srv is interface for services
- prioritize use predefined interfaces
- sometimes is needed define custom ones

workflow

- define msg into `msg`
- define srv into `srv`
- modify `CMakeLists.txt`
- modify `package.xml`
- build

```
cd ~/ros2_ws
ros2 pkg create custom_interfaces --build-type ament_python
```

msg definition

- go to `~/ros2_ws/custom_interfaces/msg`
- edit `Num.msg`

```
int64 num
```

- edit `Sphere.msg`

```
# use message interface from another package
geometry_msgs/Point center
float64 radius
```

srv definition

- go to `~/ros2_ws/custom_interfaces/srv`
- edit `AddThreeInts.msg`

```
int64 a
int64 b
int64 c
---
int64 sum
```

- needed to generating language-specific code
- interfaces rely on `rosid_default_generators`

```
find_package(geometry_msgs REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/Num.msg"
  "msg/Sphere.msg"
  "srv/AddThreeInts.srv"
  # add packages that avove messages depend on
  DEPENDENCIES geometry_msgs
)
```

package.xml

- add the following lines

```
<depend>geometry_msgs</depend>
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

build

```
# build
colcon build --packages-select custom_interfaces

# add commands to overlay
source install/setup.bash
```

```
#----- terminal 1 ----- #
ros2 interface show custom_interfaces/msg/Num
#----------------------
int64 num

ros2 interface show custom_interfaces/msg/Sphere
#----------------------
geometry_msgs/Point center
        float64 x
        float64 y
        float64 z
float64 radius
```

```
ros2 interface show tutorial_interfaces/srv/AddThreeInts
#----------------------
int64 a
int64 b
int64 c
---
int64 sum
```

use

- import in nodes that use them

```
from custom_interfaces.msg import Num
from custom_interfaces.srv import AddThreeInts
```

- add depencies into `package.xml`

```
<exec_depend>custom_interfaces</exec_depend>
```